



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Sales Predictions with XGBoost Ensemble and Iterative Forecasting

TESI DI LAUREA TRIENNALE IN
MATHEMATICAL ENGINEERING - INGEGNERIA MATEMATICA

Author: **Carlo Siniscalchi**

Student ID: 10828079

Advisor: Prof. Alessandro Toigo

Academic Year: 2024-2025

Abstract

This thesis, based on a project commissioned by Piccolo Teatro di Milano, describes the end-to-end integration of a framework for forecasting show-ticket sales trends, with the aim of supporting advertising-investment decisions.

First, a pipeline was developed to merge and clean the various datasets provided by the theatre, enriching them with additional features useful for prediction. Next, a bootstrap ensemble of XGBoost regressors was trained, with its hyperparameters optimized via Bayesian optimization, thus enabling the generation of confidence intervals for the forecasts.

Finally, using iterative forecasting, sales estimates were produced for each show, and the model's performance was evaluated using the following metrics: MSE, MAE, Prediction Interval Coverage Probability, and Interval Score.

Keywords: XGBoost, Iterative Forecasting, Sales Prediction, Trend Prediction

Abstract in lingua italiana

Questa tesi, basata su un progetto richiesto dal Piccolo Teatro di Milano, descrive l'integrazione end-to-end di un framework per la predizione dei trend di vendita degli spettacoli, con l'obiettivo di supportare le decisioni di investimento pubblicitario.

Inizialmente è stata realizzata una pipeline in grado di unire e pulire i diversi dataset forniti dal teatro, arricchendoli con feature aggiuntive utili alla previsione. Successivamente è stato addestrato un ensemble bootstrap di regressori XGBoost, i cui iperparametri sono stati ottimizzati attraverso Bayesian Optimization, permettendo così di generare intervalli di confidenza per le stime.

Infine, mediante previsioni iterative, sono state prodotte le stime di vendita per i vari spettacoli e valutate le prestazioni del modello con le seguenti metriche: MSE, MAE, Prediction Interval Coverage Probability e Interval Score.

Keywords: XGBoost, Previsione Iterativa, Previsione Vendite

Contents

Abstract	i
Abstract in lingua italiana	iii
Contents	v
1 Introduction	1
2 Data Pipeline	3
2.1 Data Presentation	3
2.2 Data Ingestion	3
3 Features and Targets	7
3.1 Approach	7
3.2 Calculate Features and Targets	7
3.3 Managing Features	10
3.4 Target	12
4 Model Training	15
4.1 Hyperparameter Tuning via Bayesian Optimization	16
4.2 Bootstrap Ensemble Training	16
5 Model Performance	19
6 Conclusions and future developments	23
Bibliography	25
Acknowledgements	27

1 | Introduction

The prose theater "Piccolo Teatro" had the need to integrate a forecasting model into a Power BI dashboard to analyze show ticket purchase trends. In particular, the marketing team wants to decide how much to invest in advertising for a specific show based on the sales curve predicted by the model. First of all it is possible to find all the project code in this [GitHub repository](#).

The available data consists of three tables. The first contains box office transactions, including information such as transaction date and time, amount spent, purchased item, and quantity. The second table contains information about the shows, such as name, performance dates (each show has multiple performances), and seating capacity. Finally, the third table reports the start and end dates of each theater season, which is useful since the start of ticket sales for all shows coincides with the beginning of the season.

To clean these data, I created three classes, each representing one table. Each class has specific methods that clean the raw input table. For example, for the transaction table I implemented a method that removes unnecessary rows, such as those related to subscription purchases, and another that assigns the correct data types to columns (e.g., the amount spent is a float). In this way, the data can be easily cleaned also in production.

Once I obtained clean and organized data, I proceeded to create the dataset to be used for training the model. I then merged the information from the various tables. In particular, on the transaction table I applied a grouping by day and show ID, so that each row contains daily aggregated information for each show. For instance, each row of the resulting table reports the total number of tickets sold on a given day.

At this stage, I created the features useful for the model training, such as cumulative revenue, the percentage of sold tickets out of the available total, moving averages, and shifted values over different periods.

By observing the data, I noticed that the ticket purchase trend tends to grow exponentially. For this reason, I applied a $\log_{1p}$ transformation to the features related to purchase trends. As the target variable, I selected the percentage of tickets sold with respect to the total on the following day, also transformed with $\log_{1p}$.

Since my goal is to predict the entire sales trend and not just the following day, I adopted an iterative forecasting approach: with the features of day t I predict the target for day $t+1$, which is then used to update the features for predicting day $t+2$, and so on.

To manage the features—both for implementing iterative forecasting and for easily enabling/disabling them, I created a Feature variable with two attributes: `enabled`, which indicates whether the feature is active, and `update`, which is a function.

The update function is called at each iteration of the iterative forecasting process to update the feature using the most recent prediction.

I chose XGBoost as the model. Since I wanted to obtain confidence intervals for the predictions, but XGBoost does not natively provide them, I adopted the bootstrapping technique: 70 models were trained, and for each prediction 70 forecasts are generated, from which I compute the confidence intervals using a Student's t distribution. The model parameters were optimized using BayesSearch.

The predictions obtained with this method are very accurate, but the confidence intervals, which should theoretically provide 90% coverage, actually cover only about 70

2 | Data Pipeline

All the code I will talk about in this section is inside `/piccolo_teatro/data_pipeline` folder.

2.1. Data Presentation

D_SALES_LIST_SALES This table contains information about each transaction at the theater. Each row corresponds to one transaction and includes:

- the date and time of the transaction,
- the total amount spent,
- the item(s) purchased,
- the quantity purchased.

D_CONFIG_PROD_LIST This table contains information about theater performances. Each row corresponds to a single performance and includes:

- the date and time of the performance,
- the hall (the theater has multiple halls),
- the show's name,
- the number of available seats.

stagioni This table contains the start and end dates of each theater season. It is particularly relevant because ticket sales open at the beginning of the season and close on the day of each performance.

2.2. Data Ingestion

In the [ingestion.py](#) file I defined 3 classes (Sales, Products, Seasons) one for each table, this way it's easier to manage and clean data. Indeed each class takes as input a Pandas

Dataframe and has specific methods to manage the Dataframe. For example inside the Sales class we have the following methods:

- **rename_cols**: Renames columns to easily usable names, removing any leading space.
- **assign_types**: Standardizes the decimal separator in the **gain** column (sometimes “,”, other times “.”), drops rows with missing values, and converts data types (float, int, datetime, str).
- **clean_cols**: Reduces the DataFrame only to the columns that are necessary for analysis.
- **clean_rows**: Removes unwanted seasons and filters to only ticket-sale transactions (e.g., excludes subscription purchases).
- **assign_index**: Sets the **date** column as the DataFrame index.
- **sort_values**: Sorts the DataFrame chronologically by **date**.
- **get_head(offset)**: Keeps only the initial fraction of rows defined by *offset*, simulating the first N days of sales.
- **get_same_show(show_id)**: Filters the DataFrame for a specific show.
- **get_groups**: Returns a **GroupBy** object grouped by **show_id** for aggregate analysis.
- **clean**: Runs **assign_types**, **clean_rows**, and **clean_cols** in sequence for a full cleaning pipeline.

Dataframes after ingestion:

- **Sales DF:**

Shape: 553171 rows × 8 cols

Column	Type
individuali_gruppi	object
online_offline	object
gain	float64
tickets	int64
date	datetime64[ns]
season_id	int64
show_id	int64
performance_id	int64

- **Shows DF:**

Shape: 7355 rows × 14 cols

Column	Type
season_name	object
D_CONFIG_PROD_LIST_T_SEASON_ID	int64
show_id	int64
D_CONFIG_PROD_LIST_PRODUCT_STATE	object
D_CONFIG_PROD_LIST_T_PERFORMANCE_ID	int64
performance_day	object
space	object
D_CONFIG_PROD_LIST_T_SPACE_ID	float64
D_CONFIG_PROD_LIST_LOG_CONFIG	object
D_CONFIG_PROD_LIST_T_LOGCONFIG_ID	float64
performance_state	object
max_tickets	int64
D_CONFIG_PROD_LIST_PRODUCT_EXTERNAL_NAME	object
show_type	object

- **Seasons DF:**

Shape: 11 rows × 6 cols

Column	Type
season_id	int64
season_name	object
inizio_vendite	datetime64[ns]
fine_vendite	datetime64[ns]
inizio_stagione	datetime64[ns]
fine_stagione	datetime64[ns]

3 | Features and Targets

3.1. Approach

Since feature calculation can be time-consuming and the final set of features and targets is not known in advance, we compute all potentially useful features from the raw data once and then use a configuration file to select the subset to feed into the model. We start with three separate cleaned tables that are not yet related: one containing show information (eg. type, number of performances), another containing transaction records, and a third listing seasons. These tables are the same as before, obtained after the ingestion process. Our goal is to merge these into a single table where each row represents a show on a specific day, enriched with both historical transaction metrics and static show metadata.

3.2. Calculate Features and Targets

The code for this section can be found in [transformation.py](#).

First, we group the sales data by `show_id`, `season`, and `day`, summing all transactions for each show on each day of a season. For instance, now each entry in the `gain` column represents the total revenue generated by a show on that specific day. From this aggregated data, we compute the following features:

- **Sales Metrics and Targets:**

Features marked with `*` can serve as prediction targets when calculated one day after the data point.

- `*gain_cum_sum`: cumulative daily revenue.
- `*tickets_cum_sum`: cumulative daily ticket sales.
- `{colName}_mvg_avg_{period}`: rolling-window moving averages over `{period}` days of `gain_cum_sum`, `tickets_cum_sum`, and `percentage_bought`.
Justification: It helps reduce noise and compare short-term growth with medium-term growth.

- **{colName}_shifted_{period}**: lagged values over {period} days of `gain_cum_sum`, `tickets_cum_sum`, and `percentage_bought` (e.g., the value of `gain_cum_sum` six days prior).
- **{colName}_delta_{period}**: delta between current value and {period} days ago of `gain_cum_sum`, `tickets_cum_sum`, and `percentage_bought`.
- **remaining_tickets**: tickets still available.
Justification: it provides the model with context on ticket scarcity, which can signal certain types of growth.
- ***percentage_bought**: proportion of tickets sold to date relative to the total available.
Justification: normalizing ticket sales allows the model to compare trend patterns across shows of different sizes.

- **Show Metadata:**

Static information about each show.

- **show_type**: e.g., children's show, cultural collaboration.
- **show_capacity**: total tickets available across all
- **num_performances**: total number of performances for the show.

- **Temporal Distances:**

Justification: Measures of position within the sales calendar.

- **start_sales_distance:** days since the start of ticket sales.
- **end_season_distance:** days remaining until season close.
- **end_sales_distance:** days remaining until the show's final performance.
- **sales_duration:** total days tickets are on sale.
- **percentage_sales_day:** fraction of elapsed sales days relative to **sales_duration**.

Justification: normalizes temporal position allows the model to compare trend patterns across shows of different sizes.

We can now plot the trend to be predicted, spanning from the first day of sales to the last.

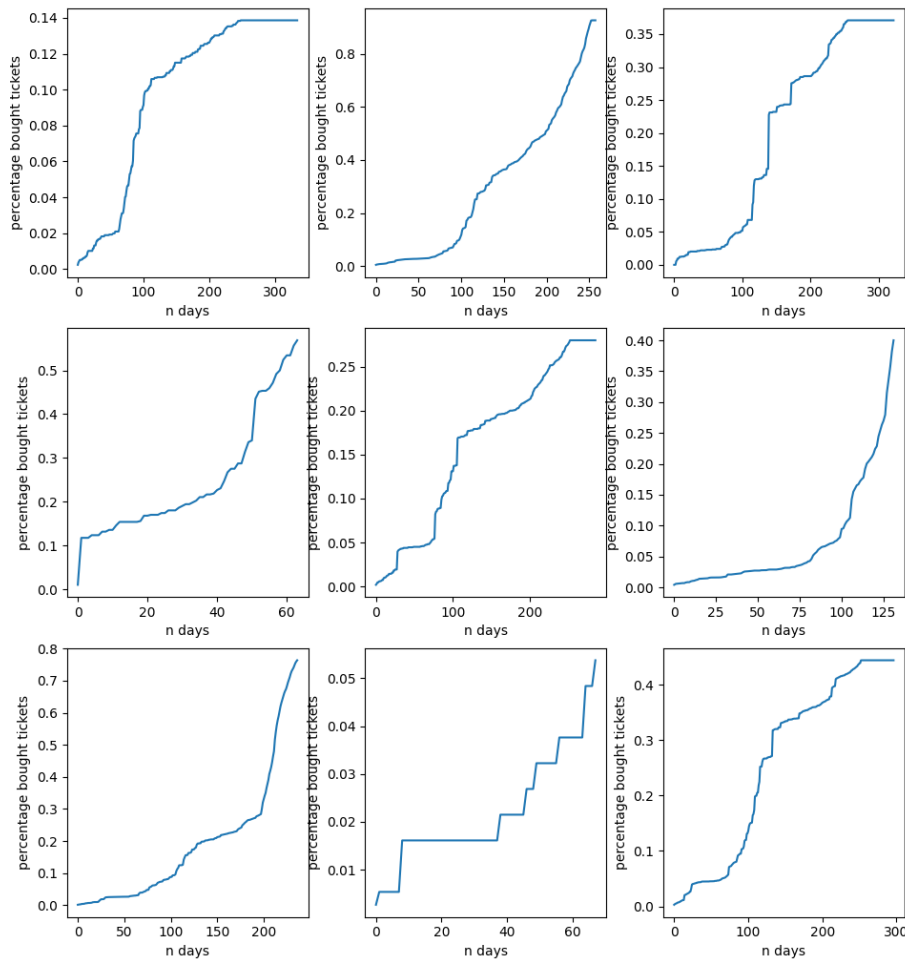


Figure 3.1: The sales trends we want to predict

Since some trends grow exponentially, we may want to apply a $\log_{1p}$ transformation:

$$\log_{1p}(x) = \ln(1 + x)$$

3.3. Managing Features

The code of this section can be found in `/config/__init__.py`. We now have more features than we actually need to train our model. To keep things simple at first, and only add complexity when necessary, we require an easy way to enable or disable individual features. In order to flexibly enable or disable individual feature transformations and to support iterative forecasting, we implement a small framework based on a class called **Feature** class. Iterative forecasting consists of using the features of day t to predict the target for day $t+1$, which is then used to update the features for predicting day $t+2$, and so on. This framework encapsulates:

- Which columns each feature concerns. There can be more than one, since for example all moving averages with different periods belong to the same feature class.
- Whether the feature is constant (does not change over time) or variable,
- Whether it is currently enabled,
- A callable `update` method to compute its new value.

```
class Feature(BaseModel):
    columns: List[str]
    const: bool
    enabled: bool
    update: Callable = None
```

We also define a class called **TimeSeriesEngine**, which takes as input a **DataFrame** containing various features. This **DataFrame** represents the known data that the model will use to make predictions and subsequently update with the predicted values. In its `__init__` method, we initialize each feature using the **Feature** class. We then add methods to update these features dynamically. This design makes it straightforward to implement iterative forecasting. We will use the following method: After each time-step prediction, we call `update_features(prediction)`. It:

1. Stores the new prediction,

2. Calls each variable feature's `update` to compute its value for the next row,
3. Calls each constant feature's `update` (simply copying the last known value),
4. Concatenates the newly computed row to the data frame.

At each iteration, a new row is added to the `DataFrame`, initially containing only known data provided as input to the `TimeSeriesEngine` class. This row contains the updated feature values along with the new prediction.

Pseudocode:

```
def update_features(self, prediction):
    self.new_prediction = prediction
    self.new_row = {}

    # Compute variable features
    for feature in self.variable_features:
        feature.update()

    # Copy constant features
    for feature in self.const_features:
        feature.update(feature.columns[0])

    # Append to DataFrame
    self.new_row = pd.DataFrame(self.new_row)
    self.df = pd.concat([self.df, self.new_row], ignore_index=True)
```

Example of an update function.

```
def __update_ma(self):
    # Concatenate hystorical data (known+previous predictions) and
    ↪ the new prediction
    values = self.df[f"{self.target}"].tolist() +
    ↪ [self.new_prediction]
    s = pd.Series(values)
    # Compute rolling mean for each period
    for period in self.periods:
```

```
avg_val = s.rolling(period).mean().iloc[-1]
self.new_row[f"{self.target}_avg_{period}"] = avg_val
```

Iterative forecasting pseudocode:

```
def predict_trend(known_df, model, last_date):
    ts_engine = TimeSeriesEngine(df=known_df)
    predictions = []
    while(current_day < last_date):
        current_day += pd.Timedelta(days=1)

        input_row = ts_engine.df.iloc[[-1]]
        # Get model prediction
        prediction = model.predict(input_row)[0]
        predictions.append(prediction)

        # Updating all the features with the new prediction
        # (ts_engine.df is updated)
        ts_engine.update_features(prediction)
```

3.4. Target

Our objective is to forecast the sales curve of a show (sales of all performances of a show). To do this, we could consider various target variables. The first that come to mind are:

- Cumulative number of tickets sold
- Cumulative revenue from ticket sales

The problem with these targets is that they are not normalized: their range varies greatly depending on the show's capacity and ticket prices. Without normalization, sales curves from different shows are not comparable, making it harder for the model to learn. Relative progress (e.g., "50% of tickets sold in the last week") can be similar across shows, but absolute ticket counts or revenue figures obscure these patterns.

Therefore, we use as our target the *cumulative percentage of tickets sold*, transformed with *log1p* (since, as we observed in the previous plot, the trends tend to grow expo-

nentially). One might consider using the day-over-day change in tickets sold, but this yields a noisy target: daily sales can fluctuate wildly due to external factors (weekends, marketing campaigns, press coverage, weather), resulting in discontinuous spikes. In contrast, the cumulative percentage curve is monotonic, smoother, and directly comparable across different shows, which facilitates model learning. Also, for Bayesian search we will use cross-validation; in particular, we will ensure that each show's data remains within a single fold.

4 | Model Training

The code for hyperparameter tuning and for training the bootstrap ensemble can be found at [/train/bootstrap_ensemble.py](#). The code for iterative forecasting can be found at [/trend_simulation/__init__.py](#). We require a single global model capable of forecasting the sales trend for any show—including those not seen during training—so we discard univariate time-series models such as ARIMA. Instead, we opt for the XGBoost regressor, a supervised ensemble of decision trees trained via gradient boosting. XGBoost [1] can learn complex nonlinear patterns, making it well suited to our task. Its drawback is the lack of native prediction intervals; to estimate uncertainty, for this reason we will employ bootstrapping [2]. When splitting our data into training, validation, and test sets, we ensure that each dataset contains entire shows to prevent data leakage. We trained our model using:

- **Target:** `percentage_bought_log1p`
- **Features:**
 - `sales_duration`
 - `start_sales_distance, end_sales_distance`
 - `percentage_sales_day, percentage_bought_log1p`
 - `percentage_bought_log1p_avg_2, ..., percentage_bought_log1p_avg_30`
 - `percentage_bought_log1p_delta_2, ..., percentage_bought_log1p_delta_30`
 - `percentage_bought_log1p_shifted_2, ..., percentage_bought_log1p_shifted_30`
- **MA and Lag Periods:** `[2, 4, 6, 8, 10, 15, 20, 30]`

I used all the previously listed features related to sales and temporal distances (Section 3.2), while I did not include show metadata because preliminary testing showed that they were not used by the model and did not improve performance.

4.1. Hyperparameter Tuning via Bayesian Optimization

First, we tune hyperparameters via Bayesian optimization [3]. We search over the following configuration space:

```
param_space: dict = {  
    'n_estimators': Integer(100, 300), # Number of trees  
    'max_depth': Integer(3, 10), # Maximum depth of each tree  
    'learning_rate': Real(0.01, 0.3, prior='log-uniform'), # Step size  
    'subsample': Real(0.5, 1.0), # Fraction of training data to sample  
    ↪ for each tree  
    'colsample_bytree': Real(0.5, 1.0) # Subsample ratio of columns when  
    ↪ constructing each tree  
}
```

4.2. Bootstrap Ensemble Training

Now that the optimal parameters have been found, the models can be trained. Using bootstrapping, the models are not trained on completely different datasets. Instead, from the training dataset, data are randomly sampled with replacement — meaning that each show may be selected multiple times — until a dataset of N shows is formed, where N is the total number of shows in the training set.

When forecasting the trend of a show, the ensemble of models produces predictions that are then aggregated. For each day the mean prediction and the 90% confidence intervals for the mean are computed. These intervals are built under the simplifying assumption that the bootstrap predictions are i.i.d., since resampling introduces sufficient variability. The Student's t distribution is used for this purpose, as the sample is finite and the variance is estimated. With more than 30 samples, the result is practically equivalent to the normal distribution, whereas in the preliminary phases fewer than 30 samples were used.

This will give us three curves: the average prediction, the lower bound, and the upper bound. We use 70 models, so each time we predict a point on the trend line we actually generate 70 forecasts, from which we compute the mean prediction as well as the lower and upper bounds. It should be noted that bootstrapping was used to obtain the confidence

intervals, since they are not natively available in XGBoost.

Pseudocode:

```
def ensemble_prediction(ensemble, input_df):  
    # 1. Collect each model's full predicted trend  
    trends = []  
    for model in ensemble:  
        pred = predict_trend(model, input_df)  
        trends.append(pred)  
    # 4. confidence interval  
    lower, upper = get_ci(trends, 0.9)  
    # 3. mean trend  
    mean = np.mean(trends, axis=0)  
  
    return lower, mean, upper
```


5 | Model Performance

Now we can see how the models perform on unseen data. First, however, we define the following metrics:

Let y_t denote the true value at time t , $\hat{y}_t$ the point forecast, and $[\ell_t, u_t]$ the associated prediction interval at time t , where $t = 1, \dots, n$ and n is the total number of forecasts.

1. Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{n} \sum_{t=1}^n (y_t - \hat{y}_t)^2$$

Justification: Errors are squared, so larger errors are penalized more heavily.

2. Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{n} \sum_{t=1}^n |y_t - \hat{y}_t|$$

Justification: Errors are not squared, each error carries the same weight.

3. Prediction Interval Coverage Probability (PICP)

$$\text{PICP} = \frac{1}{n} \sum_{t=1}^n \mathbf{1}(\ell_t \leq y_t \leq u_t)$$

Justification: It shows how often values fall within the confidence intervals.

4. Interval Score

$$\text{IntervalScore} = \frac{1}{n} \sum_{t=1}^n \left[(u_t - \ell_t) + \frac{2}{\alpha} (\ell_t - y_t) \mathbf{1}(y_t < \ell_t) + \frac{2}{\alpha} (y_t - u_t) \mathbf{1}(y_t > u_t) \right]$$

where α is the nominal miscoverage level (e.g. $\alpha = 0.05$ for a 95 percent interval).

Justification: It reflects the intervals' coverage quality, but larger intervals are penalized more.

We compute these metrics for all shows over various forecasting horizons (using the naming convention `metric_period`, where `_tot` denotes predictions up to each show's last available date).

For the test set we get (rounding at the 3th decimal):

MSE_5	MAE_5	PICP_5	IS_5	MSE_10	MAE_10	PICP_10	IS_10
4.5e-5	0.004	0.778	0.012	0.1e-3	0.007	0.732	0.023

MSE_15	MAE_15	PICP_15	IS_15	MSE_20	MAE_20	PICP_20	IS_20
0.2e-3	0.01	0.706	0.033	0.4e-3	0.013	0.694	0.045

MSE_25	MAE_25	PICP_25	IS_25	MSE_30	MAE_30	PICP_30	IS_30
0.001	0.016	0.684	0.056	0.001	0.019	0.672	0.068

MSE_tot	MAE_tot	PICP_tot	IS_tot
0.016	0.081	0.693	0.306

For the training set:

MSE_5	MAE_5	PICP_5	IS_5	MSE_10	MAE_10	PICP_10	IS_10
4.5e-5	0.004	0.757	0.013	0.1e-3	0.006	0.754	0.021

MSE_15	MAE_15	PICP_15	IS_15	MSE_20	MAE_20	PICP_20	IS_20
0.2e-3	0.008	0.743	0.031	0.4e-3	0.011	0.735	0.041

MSE_25	MAE_25	PICP_25	IS_25	MSE_30	MAE_30	PICP_30	IS_30
0.001	0.014	0.73	0.052	0.001	0.017	0.728	0.063

MSE_tot	MAE_tot	PICP_tot	IS_tot
0.013	0.073	0.747	0.277

We observe that the model achieves good results in terms of MSE and MAE on the test set, which are not significantly different from those obtained on the training set. This indicates that the model is not overfitting and is able to generalize well.

As expected, both MSE and MAE increase with the forecasting horizon, due to the accumulation of error in the iterative forecasting process. However, the growth of the error remains controlled and does not escalate excessively.

With regard to the metrics used to assess the quality of the prediction intervals, the results for PICP and IS are less satisfactory. The Prediction Interval Coverage Probability (PICP) ranges from approximately 0.69 to 0.78, whereas we would ideally expect values close to 0.90, since the intervals are constructed for a 90% confidence level. Moreover, the Interval Score (IS) increases over longer horizons, indicating that the intervals widen as uncertainty accumulates through iterative forecasting. This trend becomes particularly evident when plotting the forecasted trajectories over time.

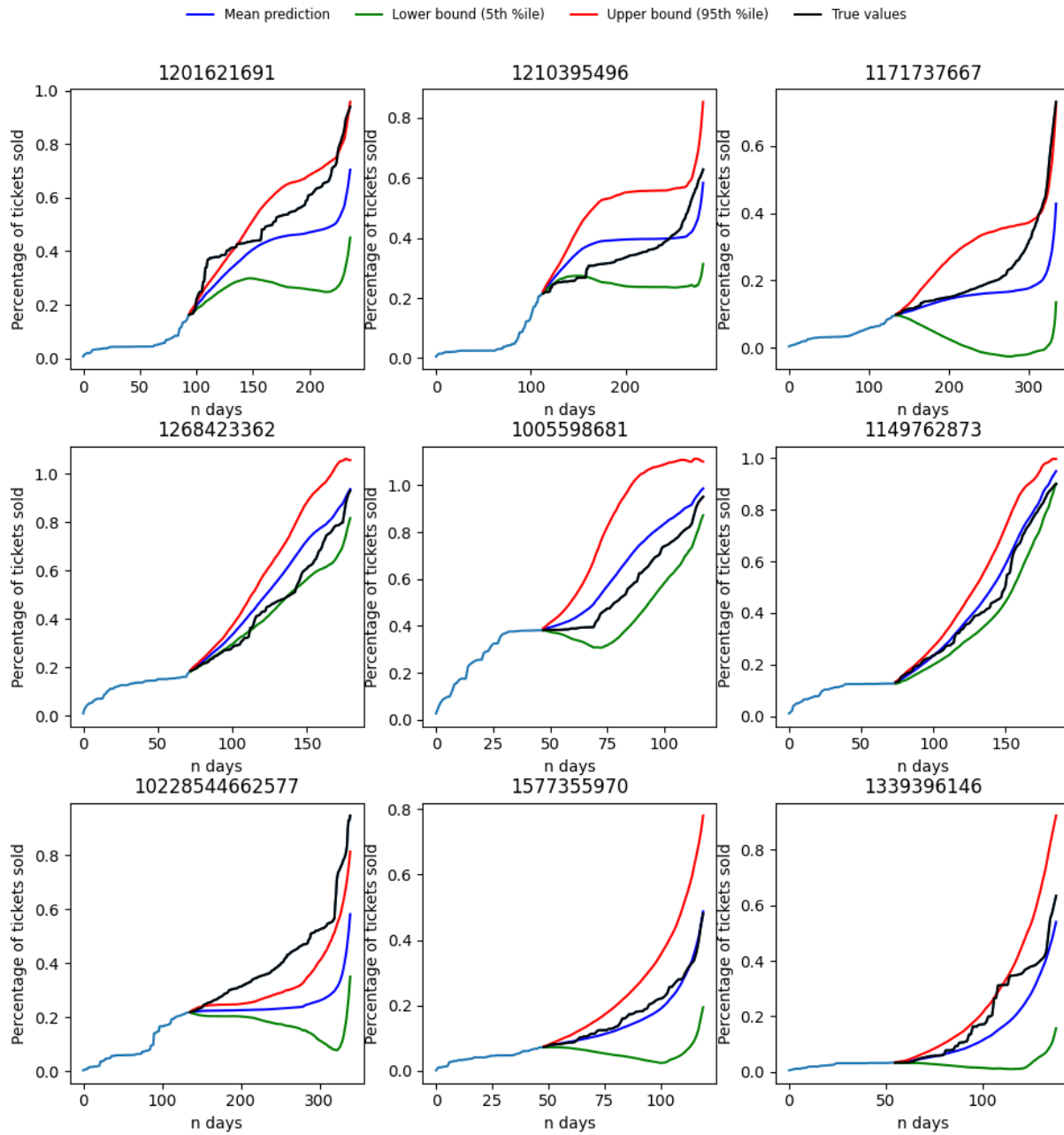


Figure 5.1: Test set predictions (the number above each graph is the show id)

6 | Conclusions and future developments

The experimental results show that an XGBoost model, combined with Bayesian hyperparameter optimization and iterative forecasting, achieves good predictive accuracy.

However, a significant limitation lies in the lack of calibration of the prediction intervals.

One possible explanation for this issue could be the low variability among the ensemble models. In the current setup, all models use the same hyperparameter configuration, derived from Bayesian optimization. To address this, an alternative strategy could be to assign random hyperparameters to each model or inject noise into the training data to further diversify the ensemble. Finally, it is also possible that the number of models in the ensemble is still insufficient. Increasing the ensemble size may lead to more reliable confidence intervals.

Bibliography

- [1] T. Chen and C. Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. doi: 10.1145/2939672.2939785. URL <https://doi.org/10.1145/2939672.2939785>.
- [2] B. Efron. Bootstrap methods: another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979. doi: 10.1214/aos/1176344552. URL <https://doi.org/10.1214/aos/1176344552>.
- [3] J. Snoek, H. Larochelle, and R. P. Adams. Practical bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems*, volume 25, pages 2951–2959, 2012. URL <https://papers.nips.cc/paper/4522-practical-bayesian-optimization-of-machine-learning-algorithms.pdf>.

Acknowledgements

Desidero innanzitutto ringraziare il Prof. Alessandro Toigo per la costante disponibilità e la grande gentilezza dimostrate durante tutto il percorso di questa tesi.

Un grazie enorme alla mia famiglia, che mi ha permesso di studiare e mi ha sempre sostenuto.

Grazie a Lucia, che in questi tre anni mi ha accompagnato e supportato in ogni momento.

E grazie anche ad Alessandro, che mi ha sopportato con la sua infinita pazienza.

